Пользовательская инструкция

AI Платформа AIналитик

Скачать: https://github.com/chaussky/ainalyst.git

Глава 7 — Requirements Analysis and Design Definition

Общая характеристика Главы 7

Глава 7 BABOK — «Requirements Analysis and Design Definition» — это переход от стратегии к конкретике. Если Глава 6 ответила на вопросы «почему меняемся» и «какова стратегия», то Глава 7 отвечает на вопрос: **что именно разрабатываем, как это организовано, правильно ли написано, нужно ли бизнесу, как будем реализовывать и стоит ли это делать вообще.**

Шесть задач Главы 7 выстраиваются в единую логику:

```
7.1 Спецификация   →   7.2 Верификация (правильно написано?)
          ↓                          ↓
  7.4 Архитектура    →   7.3 Валидация (нужно бизнесу?)
          ↓
  7.5 Варианты дизайна   →   7.6 Оценка ценности и рекомендация
```

Это самая «производящая» глава проекта: именно здесь появляются требования — User Stories, функциональные требования, бизнес-правила, модели данных, бизнес-процессы. Именно здесь они проверяются на качество и ценность. И именно здесь формируется финальная рекомендация спонсору: какой вариант реализации выбрать и почему.

Почему эту главу часто делают плохо. BA нередко сводит Главу 7 к одному действию: написать список функциональных требований в таблице Excel. Верификация не проводится — «и так понятно». Валидация не проводится — «заказчик сам сказал что хочет». Архитектура требований не строится — «разберёмся по ходу». В результате разработчики получают список функций без контекста, без приоритетов, без объяснения «зачем», с противоречиями и пробелами — и проект начинает буксовать уже на первом спринте.

Фаза проекта. Глава 7 выполняется в фазе `design`. Если вы ещё не переключили фазу — попросите AIналитика или запустите `python phase.py design`.

Задача 7.1 — Specify and Model Requirements (Спецификация и моделирование)

Краткое описание

BA переводит подтверждённые результаты выявления из Главы 4 в формальные спецификации: User Stories, функциональные требования, бизнес-правила, модели данных (ERD), описания бизнес-процессов (BPMN). Каждый созданный артефакт автоматически регистрируется в трассировочном репозитории (5.1) и становится частью живого реестра требований проекта.

Боли и проблемы BA

«Требования в голове, а не в документе». BA провёл 15 интервью, всё понял — и начал «передавать» знания разработчикам на встречах. Через месяц у разных членов команды разное понимание одного и того же требования. Нет единого источника правды.

«Один большой список FR». Всё записано как «Система должна...» в одну таблицу: и бизнес-процессы, и правила, и пользовательские сценарии, и технические ограничения — вперемешку. Разные стейкхолдеры читают этот список и видят разное. Никто не видит целостной картины.

«Требования написаны размыто». «Система должна быстро обрабатывать запросы», «должна быть удобным интерфейс», «необходимо обеспечить безопасность» — эти формулировки нельзя протестировать, нельзя оценить. Разработчик интерпретирует их по-своему, QA — по-своему, заказчик — ещё иначе.

«Трассировки нет». Требование написано, но никто не знает откуда оно взялось и зачем. Когда спонсор спрашивает «а зачем нам FR-047?» — BA не может ответить. Когда стейкхолдер предлагает «давайте уберём вот это» — нет способа быстро понять что за этим тянется.

«Нет проверки покрытия». BA написал 50 требований и думает что готово. Но три из шести бизнес-целей не покрыты ни одним требованием — просто потому что в потоке работы об этом забыли.

Что мы реализовали

Анализ контекста выявления как стартовая точка. Первый шаг — `analyze_elicitation_context`: платформа читает артефакты 4.3 (подтверждённые результаты выявления) и предлагает список требований-кандидатов с рекомендуемым типом и приоритетом. BA не начинает с чистого листа — видит структурированный список отправных точек на основе того, что уже было зафиксировано в Главе 4.

Шесть типов артефактов — каждый для своей цели. Платформа поддерживает полный спектр нотаций:

- **User Story** — пользовательский сценарий в формате «Как [роль], я хочу [действие], чтобы [выгода]» с Acceptance Criteria
- **Functional Requirement (FR)** — системное поведение в формате «Система должна...»

- **Non-Functional Requirement (NFR)** — ограничения на качество: производительность, безопасность, доступность
- **Business Rule (BR)** — правила предметной области, которым система должна следовать
- **Business Process (BP)** — BPMN-модель процесса с автогенерацией Activity Diagram (.puml)
- **Data Dictionary + ERD** — описание сущностей и их связей с автогенерацией диаграммы

Автоматическая регистрация в трассировке. Каждый созданный артефакт немедленно появляется в репозитории 5.1 со статусом `draft`. Трассировка к бизнес-целям встраивается в процесс создания — BA указывает linked goals при создании требования, а не добавляет их потом отдельным усилием.

Матрица покрытия в конце. `build_coverage_matrix` показывает: какие бизнес-цели покрыты требованиями, а какие нет. Зелёный — покрыто, красный — ни одного требования, жёлтый — более 10 требований (возможный over-engineering). BA видит пробелы явно, до того как перейдёт к следующей задаче.

Ценности для BA

От «я всё понял» к «всё зафиксировано». Знание, которое живёт в голове BA, — это риск. Платформа превращает это знание в структурированные артефакты с трассировкой и историей. Новый участник команды, пришедший через месяц, видит полный контекст: откуда требование, зачем, кто источник.

Разные артефакты для разных аудиторий. User Story читает заказчик — он понимает. FR читает разработчик — он понимает. BR читает юрист — он понимает. Это не один документ «для всех» — это правильный тип артефакта для правильного стейкхолдера. Платформа помогает выбрать тип, а не оставляет BA угадывать.

Автодиаграммы без инструментов. Business Process и ERD генерируют PlantUML-диаграммы автоматически. BA не открывает Visio, не рисует вручную — описывает логику текстом, платформа генерирует визуализацию. Это экономит время и снижает вероятность расхождения между документом и диаграммой.

Покрытие бизнес-целей как критерий завершённости. «Готово» теперь означает не «я написал всё что вспомнил», а «все бизнес-цели покрыты требованиями». Матрица покрытия даёт объективный ответ на вопрос о готовности.

Как пользоваться: пример

Продолжение проекта contract-approval. Выявление (Глава 4) завершено, Глава 6 выполнена.

«Проанализируй контекст выявления для проекта contract-approval.»

Платформа читает 4.3-артефакты и предлагает: 3 User Story (согласующие роли), 7 FR (статусы, уведомления, интеграция), 2 BR (правила эскалации), 1 BP (процесс согласования), 1 ERD (структура договоров).

«Создай User Story: как Согласующий, хочу получать уведомление о новом договоре на согласовании, чтобы не пропустить дедлайн. AC: уведомление приходит в течение 5 минут после назначения; содержит ссылку на договор и дедлайн.»

US-001 создана, зарегистрирована в 5.1, привязана к BG-001.

«Проверь покрытие бизнес-целей.»

BG-001 покрыта 4 требованиями. BG-002 (прозрачность статусов) — ни одного. BA создаёт дополнительные требования.

Задача 7.2 — Verify Requirements (Верификация требований)

Краткое описание

BA проверяет написанные требования на качество формулировок: атомарность, однозначность, тестируемость, полноту, согласованность. Это проверка «правильно ли написано», а не «правильное ли это требование» — для последнего есть задача 7.3.

Верификация — это профессиональный контроль качества до того, как требования уйдут к разработчикам и тестировщикам. Ошибку в формулировке, найденную здесь, исправить стоит минут. Ту же ошибку, найденную в разработке — дни.

Боли и проблемы BA

«И так понятно». Самая частая причина пропустить верификацию. Но «понятно BA» и «понятно разработчику» — разные вещи. «Система должна быстро отвечать» — BA имеет в виду секунды, разработчик — «ну, не висеть». При первом же баге начинается выяснение «что вы имели в виду».

Нетестируемые требования. «Система должна обеспечивать высокий уровень безопасности» — что это значит? Как тестировщик проверит это требование? Никак. Такие требования создают иллюзию полноты спецификации, не добавляя реальной ценности.

«Одно требование — три функции». «Система должна принимать заявки, обрабатывать их и отправлять уведомления» — это три разных требования в одном. Разработчику непонятно как их трекать, тестировщику — как проверять по отдельности.

Расхождение между моделями. BA создал ERD с 8 сущностями и отдельно описал бизнес-процесс с другим набором сущностей. Расхождение никто не замечает — пока разработчик не спрашивает «что есть что».

«Требования согласованы» без реального согласования. Один стейкхолдер сказал «система должна поддерживать Excel», другой — «никакого Excel, только API». Оба требования записаны, противоречие никто не заметил.

Что мы реализовали

Автоматическая проверка по 9 характеристикам BABOK. `check_req_quality` анализирует все требования проекта (или выбранный список) по двум группам:

- **Группа A (автоматически):** атомарность (нет «и», «а также», «или»), однозначность (нет «быстро», «удобно», «надлежащим образом»), тестируемость (есть конкретный критерий), приоритизированность, краткость
- **Группа B (по структуре):** согласованность (нет конфликтов статусов в 5.1), полнота (есть source_artifact и трассировка)

Результат — конкретный список проблем с указанием: какое требование, какая характеристика нарушена, severity (blocker / major / minor), как исправить.

Трекер проблем верификации. Каждая найденная проблема фиксируется как Verification Issue (VI-xxx) с типом, severity и ответственным. BA не держит список правок в голове — все открытые и закрытые issues хранятся в системе. После исправления issue закрывается с notes о том что именно было исправлено.

Проверка согласованности моделей. `check_model_consistency` сравнивает артефакты из 7.1 между собой: сущности в Data Dictionary vs ERD, Use Cases vs UC-диаграмма, участники бизнес-процесса vs акторы. Расхождения выявляются автоматически — BA не держит несколько документов в голове и не ищет расхождения вручную.

Статус `verified` как явный переход. Требование с blocker-issues не может получить статус `verified` без явного решения BA. Это барьер: нельзя «забыть» про критичную проблему и двигаться дальше. После устранения всех блокеров — `mark_req_verified` переводит требование в следующий статус.

Verification Report для передачи в 5.5 и 7.3. Итоговый отчёт содержит: процент верифицированных требований, топ-проблемы по типам, список оставшихся blocker-issues, вердикт о готовности к согласованию. Это готовый документ — не нужно готовить отдельную сводку.

Ценности для BA

Ошибки найдены до разработки, а не во время. Замена «быстро» на «в течение 2 секунд при нагрузке до 500 пользователей» занимает минуту в 7.2. Та же правка в середине спринта — это обсуждение, переоценка, возможно переработка архитектуры. Стоимость исправления ошибки растёт экспоненциально с каждым этапом — платформа помогает найти её на самом дешёвом.

Тестировщик работает с чёткими критериями. Верифицированное требование содержит измеримый критерий. Это не просто аккуратность — это прямая экономия времени QA: меньше вопросов «что вы имели в виду», меньше тест-кейсов написанных «от фонаря», меньше спорных багов «это фича или дефект».

Профессиональная позиция перед стейкхолдерами. Когда BA передаёт Verification Report с 94% verified и списком закрытых issues — это демонстрация зрелого процесса. Не «мы написали требования», а «мы написали требования и проверили их по 9 критериям BABOK с документированными результатами».

Как пользоваться: пример

«Проверь качество всех требований по проекту contract-approval.»

Результат: US-003 — missing*ac (blocker), FR-007 — слово «оперативно» (ambiguity, major), FR-012 — два действия в одном требовании (not*atomic, major).

«Зафиксируй issue по US-003: нет Acceptance Criteria, blocker.»

VI-001 создан.

«US-003 исправлена: добавлено 3 AC — уведомление приходит в течение 5 минут, содержит ссылку и дедлайн, при недоступности email фиксируется в логе.»

«Закрой VI-001.» — resolution_note зафиксирована.

«Проверь согласованность моделей.» — ERD и Data Dictionary синхронизированы, расхождений нет.

«Верифицируй все исправленные требования и сгенерируй отчёт.»

Verification Report: 48/50 verified, 2 minor-issues открыты, вердикт — готово к валидации.

Задача 7.3 — Validate Requirements (Валидация требований)

Краткое описание

BA проверяет не качество формулировок (это 7.2), а содержательную правильность: нужны ли эти требования бизнесу, соответствуют ли они бизнес-целям и будущему состоянию из Главы 6, выявлены ли ключевые допущения. Это итерационная задача — она может запускаться несколько раз на разных стадиях проекта.

Боли и проблемы BA

«Заказчик сказал — значит нужно». Требование попало в спецификацию потому что кто-то его попросил. Но соответствует ли оно бизнес-целям проекта? Иногда стейкхолдеры просят то, что хотят, а не то, что нужно бизнесу. Различие критично.

«Orphan requirements». В середине проекта BA обнаруживает, что 15% требований не трассируются ни к одной бизнес-цели. Откуда они? Зачем? Исключить? Но кто-то же их написал, и теперь страшно убирать.

Неявные допущения, которые взрываются в конце. «Мы предполагали, что интеграция с ERP возможна через API» — оказывается, ERP не поддерживает API. Это допущение никто не зафиксировал и не валидировал. Теперь 30% требований нужно переписывать.

«Бизнес-контекст существует в голове BA». BA знает бизнес-цели проекта, но нигде они не зафиксированы как эталон для проверки требований. Каждая проверка — это «помню что мы хотели чего-то похожего».

Что мы реализовали

Бизнес-контекст как явный эталон. `set_business_context` фиксирует бизнес-цели (с KPI), описание будущего состояния и скоуп решения в одном месте. Если задачи 6.1 и 6.2 выполнены — данные предзаполняются автоматически из их артефактов: BA не переносит одни и те же данные вручную. Этот контекст становится эталоном для всей последующей валидации.

Автоматическая проверка соответствия бизнес-целям. `check_business_alignment` запускает BFS-обход графа трассировки 5.1 и проверяет: достижим ли из каждого верифицированного требования узел типа `business_goal`? Требования без трассировки к бизнес-целям помечаются явно — BA принимает решение: это пробел в трассировке или реально «лишнее» требование.

Реестр допущений с явным статусом. `log_assumption` фиксирует каждое предположение: что предполагается, на каком основании, насколько рискованно. Допущения с высоким риском — `high_risk` — являются предупреждением при попытке перевести требование в статус `validated`. Это побуждает BA валидировать их до перехода к следующему этапу, а не забыть.

Три оси проверки. Платформа структурирует валидацию по трём вопросам: (1) Ценность — приносит ли требование выгоду стейкхолдерам? (2) Соответствие будущему состоянию — помогает ли оно достичь to-be из 6.2? (3) Допущения — выявлены ли скрытые предположения и управляются ли они?

Validation Report как выход для 5.5. Итоговый отчёт — матрица покрытия бизнес-целей, список orphan req, статус допущений. Готовый документ для передачи на согласование в задачу 5.5.

Ценности для BA

«Лишние» требования выявлены до разработки. Требование без трассировки к бизнес-цели — это кандидат на удаление или уточнение. Найти 8 таких требований в 7.3 — экономия ресурсов разработки. Найти их после релиза — вопрос «зачем мы это сделали?» с неприятным ответом.

Допущения управляются, а не накапливаются. Реестр допущений — это не формальность. Это список «бомб», которые могут взорваться в конце. Платформа делает их видимыми и отслеживаемыми. High-risk допущение, зафиксированное и валидированное на этапе 7.3, не превращается в кризис в фазе разработки.

Связность с Главой 6 без дублирования. BA, который добросовестно выполнил 6.1 и 6.2, получает предзаполненный бизнес-контекст автоматически. Данные переиспользуются — не переносятся вручную. Это снижает риск расхождения между стратегическим документом и критериями валидации.

Как пользоваться: пример

«Установи бизнес-контекст для contract-approval.»

Если 6.1/6.2 выполнены — платформа предзаполняет: BG-001 (сократить цикл с 14 до 3 дней), будущее состояние (единый реестр статусов, автоуведомления), скоуп (модуль статусов + ERP-интеграция, без электронной подписи).

«Проверь соответствие всех верифицированных требований бизнес-целям.»

Результат: 46 из 48 трассируются к BG-001 или BG-002. FR-031 и FR-032 — orphan requirements.

«FR-031 и FR-032 — это требования к мобильному приложению. Они вне скоупа. Пометь как outofscope.»

«Зафикисируй допущение: предполагаем что ERP поддерживает REST API. Риск — high, нужно проверить с IT-архитектором до старта разработки.»

ASM-001 создано с приоритетом валидации.

«Сгенерируй Validation Report.» — готов к передаче в 5.5.

Задача 7.4 — Define Requirements Architecture (Архитектура требований)

Краткое описание

BA организует требования в целостную структуру — архитектуру: определяет точки зрения (viewpoints) разных стейкхолдеров, группирует требования в соответствующие представления (views), выявляет структурные разрывы. Результат — Architecture Document, который передаётся в 7.5 как входной артефакт и в 4.4 для коммуникации с командой.

Боли и проблемы BA

«Каждый стейкхолдер видит разное». Заказчик хочет видеть бизнес-процессы, разработчик — функциональные требования, архитектор данных — модели данных, CISO — требования безопасности. Если все 80 требований свалены в одну таблицу, каждый тратит время на поиск «своего» и рискует потерять важное.

«Никто не видит целостную картину». Требования написаны — но непонятно как они образуют систему. Нет ответа: покрыты ли все пользовательские роли сценариями? Есть ли NFR к каждому компоненту? Где разрывы?

«Архитектор получает требования без контекста». Технический архитектор, которому нужно проектировать систему, получает список из 60 FR и должен сам разбираться в структуре. Это его время, его риск ошибиться в понимании — и косвенно время и деньги проекта.

Что мы реализовали

Автоматический маппинг по типам артефактов. `analyze_requirements_architecture` читает репозиторий 5.1 и автоматически распределяет требования по стандартным точкам зрения: User Stories и Use Cases → «Пользователи и взаимодействие»; FR/NFR → «Функциональность»; BP → «Бизнес-процессы»; DD/ERD → «Данные»; BR → «Бизнес-правила». Это происходит без участия BA — платформа формирует картину из того, что уже создано в 7.1.

Кастомные viewpoints для специфических контекстов. `add_custom_viewpoint` позволяет добавить точку зрения, которой нет в стандартном наборе: «Безопасность и доступ» для банков, «Аудит и compliance» для регуляторных проектов, «Миграция данных» для проектов замены legacy-систем. BA указывает какие именно требования входят — платформа включает их в архитектуру.

Двухуровневая проверка разрывов. `check_architecture_gaps` выявляет пробелы на двух уровнях:

- Матричный уровень: есть ли у каждого стейкхолдера своя точка зрения? Покрыта ли каждая бизнес-цель хотя бы одним представлением?
- Семантический уровень: есть ли Use Case без бизнес-процесса? NFR без привязки к FR? Стейкхолдер без ни одного требования?

Снапшот архитектуры с версионированием. `save_architecture_snapshot` фиксирует текущее состояние архитектуры с версией и комментарием. История снапшотов сохраняется — можно видеть как архитектура требований развивалась по ходу проекта.

Ценности для BA

Каждый стейкхолдер получает «свой» срез. Architecture Document организован по точкам зрения. Заказчик открывает раздел «Бизнес-процессы» и видит только то, что ему нужно. Архитектор открывает «Функциональность» и «Данные». Это снижает когнитивную нагрузку на чтение и снижает риск что важное требование будет упущено нужным стейкхолдером.

Структурные разрывы найдены до дизайна. Обнаружить что пользовательская роль «Финансовый контролёр» присутствует в реестре стейкхолдеров, но не покрыта ни одним Use Case — в 7.4 это 10 минут на создание недостающего артефакта. Обнаружить то же самое в 7.5 при разработке дизайн-опций — это переработка и потеря времени.

Входной артефакт для проектирования. Architecture Document — это именно то, что нужно техническому архитектору для начала работы над дизайн-опциями (7.5). Структурированная картина вместо хаотичного списка. BA экономит время команды и снижает количество уточняющих вопросов.

Как пользоваться: пример

«Проанализируй архитектуру требований для contract-approval.»

Платформа строит картину: 4 viewpoints, 48 требований распределены. Предупреждение: BG-002 не покрыта viewpoint «Данные».

«Добавь кастомный viewpoint "Интеграция с ERP": NFR-004, FR-019, FR-020, FR-021. Стейкхолдер — IT-архитектор.»

«Проверь разрывы.»

Critical: стейкхолдер «Финансовый контролёр» в реестре 3.2, но ни одного US/UC. Warning: NFR-008 не привязан ни к одному FR.

«Зафиксируй снапшот v1.0.» — Architecture Document сохранён, готов к передаче в 7.5.

Задача 7.5 — Define Design Options (Варианты дизайна)

Краткое описание

BA формирует и сравнивает варианты реализации решения: разработать с нуля (Build), купить готовое (Buy) или комбинировать (Hybrid). Для каждого варианта распределяет требования по версиям (v1 / v2 / out*of*scope) и сравнивает варианты по взвешенным критериям. Результат — Design Options Report, который передаётся в 7.6 для финальной рекомендации.

Боли и проблемы BA

«Единственный вариант». «Мы сразу решили что будем делать Build — зачем рассматривать другое?» В результате нет анализа альтернатив, нет сравнения, нет обоснования выбора. Если что-то пойдёт не так — нет документального ответа на вопрос «а вы рассматривали другие варианты?»

«Всё в первой версии». BA включает 100% требований в MVP. В результате первый релиз задерживается, бюджет выходит за рамки, команда перегружена. Приоритизация требований по версиям — не «что хотелось бы» а «что необходимо» — один из самых болезненных разговоров с заказчиком.

«Зависимости не проверяются». BA решил, что FR-010 пойдёт в v2, но FR-010 является зависимостью для FR-003, которая в v1. В итоге разработчик не может реализовать FR-003 без FR-010. Это обнаруживается в Sprint 2 — и приводит к пересмотру плана.

«Критерии выбора — личное мнение». «Я думаю, лучше Buy — дешевле и быстрее». Почему именно Buy? Непонятно. Оппонент говорит «я думаю Build — гибче». Разговор переходит в обмен мнениями без данных.

Что мы реализовали

Три подхода с явными trade-offs. Для каждого варианта (Build / Buy / Hybrid) BA описывает компоненты, возможности для улучшения и метрики успеха. Платформа структурирует описание вариантов по единому шаблону — все три варианта сопоставимы по одним и тем же измерениям.

Автоматическое распределение требований по версиям. `allocate_requirements` в режиме `auto_suggest=True` читает приоритеты из репозитория 5.1: Must → v1, Should → v1/v2, Could → v2, Won't → out*of*scope. BA получает предложение — проверяет его, вносит переопределения для конкретных требований. Это не автоматическое решение, это автоматически подготовленная отправная точка для разговора.

Проверка зависимостей автоматически. После утверждения allocation платформа запускает BFS по графу зависимостей: если req A в v1 зависит от req B в v2 — это предупреждение. BA видит конфликт явно и принимает решение: передвинуть B в v1 или пересмотреть зависимость.

Матрица сравнения вариантов с настраиваемыми критериями. `compare_design_options` строит сравнение по стандартным критериям: стоимость, скорость, риски, покрытие требований, гибкость. BA может добавить кастомные критерии: например, «соответствие вендорской политике компании» или «возможность поэтапного внедрения». Результат — сравнительная таблица с обоснованием для стейкхолдеров.

Ценности для BA

Обоснованный выбор, а не «мы так решили». Сравнительная матрица — это документ для стейкхолдеров. Когда директор спрашивает «почему Hybrid?» — BA открывает Design Options Report: вот три варианта, вот критерии, вот оценки, вот почему Hybrid выигрывает по соотношению покрытия и гибкости при данном бюджете. Это разговор с данными.

v1 — реально реализуемая версия. Auto_suggest на основе приоритетов из 5.3 + проверка зависимостей — это защита от перегруженного MVP. BA приходит к команде разработки с allocation, где v1 содержит только Must + проверенные зависимости. Меньше неприятных сюрпризов в Sprint 1.

Design Options Report как мост к 7.6. Документ содержит все данные которые нужны для оценки ценности: варианты, компоненты, покрытие требований по версиям, potential improvements. BA не готовит отдельный документ для следующего шага — он уже создан.

Как пользоваться: пример

«Создай три варианта дизайна для contract-approval. OPT-001: Build — собственная разработка модуля. OPT-002: Buy — готовое SaaS-решение DocuWare. OPT-003: Hybrid — open-source BPM + кастомный модуль уведомлений.»

Три варианта созданы с компонентами и improvement opportunities.

«Для каждого варианта распредели требования по версиям, используй автопредложение.»

Auto*suggest: 28 req → v1, 14 → v2, 6 → out*of_scope. Проверка зависимостей: FR-019 (v2) является зависимостью для FR-015 (v1) — предупреждение.

«FR-019 перенеси в v1 для всех вариантов. Остальное принимаю.»

«Сравни варианты.»

Матрица: OPT-003 (Hybrid) выигрывает по покрытию и гибкости, OPT-002 (Buy) — по скорости, OPT-001 (Build) — по кастомизируемости. BA документирует предварительную позицию: «Рекомендую OPT-003 с финализацией в 7.6».

Задача 7.6 — Analyze Potential Value and Recommend Solution (Оценка ценности и рекомендация)

Краткое описание

BA оценивает потенциальную ценность каждого варианта дизайна — выгоды, затраты, риски — сравнивает их и формирует официальную рекомендацию спонсору. Это финальная задача Главы 7. Результат — Recommendation Document, который передаётся спонсору для принятия решения и в Главу 8 как baseline для оценки достигнутого результата.

Боли и проблемы BA

«Рекомендация — личное мнение». «Я считаю что этот вариант лучше» — это не профессиональная позиция для документа, который идёт к спонсору с бюджетом в несколько миллионов. Нет данных, нет методологии, нет обоснования.

«Выгоды не считаются — только перечисляются». «Вариант A даст нам: ускорение процессов, снижение ошибок, повышение удовлетворённости». Это список желаний, не анализ ценности. Сколько стоит каждая выгода? Насколько мы в ней уверены?

«Нет success metrics». Решение принято, проект запущен. Через год спонсор спрашивает: «Мы добились того чего хотели?» Нет ответа — потому что никто не зафиксировал что именно считается успехом до начала работ.

«no_action не рассматривается». Если ни один вариант не даёт достаточной ценности — правильная рекомендация может быть «не делать ничего» или «пересмотреть анализ». Но у BA нет инструмента для этого вывода, поэтому рекомендация всегда «делать что-нибудь».

Что мы реализовали

Структурированная оценка по формуле. `add_value_assessment` для каждого варианта собирает: типы выгод (финансовые, операционные, стратегические, человеческие) с оценкой magnitude и confidence; затраты (внедрение, операционные); рисковый профиль (читается из 6.3 автоматически, если файл есть). `compare_value` считает Value Score по формуле: `Benefits×2.0 + Alignment×1.5 - Cost×1.5 - Risk_Penalty×1.0`. Это не «чёрный ящик» — BA видит каждую составляющую.

Четыре легитимных исхода рекомендации. Платформа поддерживает все возможные выводы: `recommend_option` (один вариант явно лучше), `recommend_parallel` (два варианта реализуются параллельно), `recommend_reanalyze` (ни один не подходит — нужен новый анализ), `no_action` (выгоды меньше затрат — не делаем). BA не загнан в угол «нужно что-то рекомендовать».

Success metrics как обязательный элемент. Для рекомендации типа `recommend_option` или `recommend_parallel` — success_metrics обязательны. Это не формальность: это baseline для Главы 8. «NPS > 65 к декабрю», «Время обработки < 3 дней через 6 месяцев» — конкретные измеримые критерии успеха, зафиксированные до начала разработки.

Pre-flight проверка перед финализацией. `check_value_readiness` (опционально) проверяет полноту оценок: все ли варианты оценены, есть ли бизнес-контекст, корректны ли данные. Это страховка перед тем как документ уйдёт к спонсору.

Recommendation Document — профессиональный выходной артефакт. `save_recommendation` генерирует документ с: резюме для спонсора, оценкой каждого варианта, Value Score матрицей, обоснованием рекомендации и списком success metrics. Это готовый документ для презентации.

Ценности для BA

Рекомендация спонсору с данными, а не с мнением. Recommendation Document — это структурированный аргумент: вот три варианта, вот как мы их оценивали, вот формула, вот winner с Value Score 8.4 против 6.1 и 4.7, вот почему. Спонсор принимает решение с пониманием обоснования. BA защищён, если что-то пойдёт не так.

Success metrics = защита результата проекта. Когда через год спонсор спрашивает «мы достигли цели?» — BA открывает Recommendation Document: success metrics были «время обработки < 3 дней» и «NPS > 65». Факт: 2.8 дней и NPS 68. Проект успешен по зафиксированным критериям. Это объективная оценка, а не субъективное «кажется, стало лучше».

Передача в Главу 8 без потерь. Recommendation Document с success metrics — это точный входной артефакт для задач Solution Evaluation (Глава 8). Не нужно вспоминать «что же мы хотели достичь» — это зафиксировано в документе с датой и версией.

Профессиональный завершающий акт Главы 7. BA, который проходит весь путь 7.1 → 7.2 → 7.3 → 7.4 → 7.5 → 7.6, приходит к спонсору с полным пакетом: требования специфицированы, верифицированы, валидированы, организованы в архитектуру, варианты реализации сравнены,

ценность рассчитана, рекомендация обоснована. Это не просто список требований — это полноценный аналитический продукт.

Как пользоваться: пример

«Оцени ценность всех трёх вариантов для contract-approval.»

OPT-003 (Hybrid): выгоды — операционные (сокращение времени, снижение ошибок, confidence 0.8), стратегические (гибкость масштабирования, confidence 0.6); затраты — средние; риски подтягиваются из 6.3 (2 High-риска, но mitigation-планы есть).

«Сравни варианты.»

Value Score: OPT-003 → 8.4, OPT-002 → 6.1, OPT-001 → 5.3. Winner: OPT-003.

«Сформируй финальную рекомендацию: recommend_option, OPT-003. Success metrics: время согласования < 3 рабочих дней через 6 месяцев, adoption rate > 85% через 3 месяца.»

Recommendation Document сохранён. Проект готов к Главе 8 (Solution Evaluation) и к передаче решения в разработку.

Финальный синтез по Главе 7

Глава 7 — это производящая глава проекта. Если Глава 6 даёт стратегическое обоснование, то именно в Главе 7 создаётся интеллектуальный продукт BA: структурированные, верифицированные, валидированные требования с архитектурой, вариантами реализации и обоснованной рекомендацией. Это то, ради чего существует профессия бизнес-аналитика.

Каждая задача Главы 7 устраняет конкретный класс рисков:

- 7.1 устраняет риск «требования только в голове» — создаёт единый источник правды
- 7.2 устраняет риск «нетестируемые и размытые формулировки» — верифицирует качество
- 7.3 устраняет риск «лишние требования без ценности» — проверяет соответствие бизнес-целям
- 7.4 устраняет риск «никто не видит целостную картину» — организует требования для всех аудиторий
- 7.5 устраняет риск «перегруженный MVP и незамеченные зависимости» — структурирует варианты
- 7.6 устраняет риск «нет критериев успеха» — фиксирует success metrics до начала разработки

Цепочка артефактов Главы 7 после полного прохождения:

```
reports/
    7_2_verification_report_{project}.md    ← качество требований, issues
    7_3_validation_report_{project}.md      ← соответствие бизнес-целям, допущения
    7_4_architecture_{project}.md           ← viewpoints, разрывы, снапшот
    7_5_design_options_{project}.md         ← варианты, allocation, сравнение
    7_6_recommendation_{project}.md         ← ценность, рекомендация, success metrics
```

Спецификации требований (US, FR, NFR, BR, BP, ERD) хранятся в `governance_plans/{project}_specs/` и доступны как живые артефакты, а не как статичный документ.

Ответственность BA в Главе 7 сосредоточена на аналитических решениях: что именно является требованием, какие допущения критичны, как распределить требования по версиям, какой вариант реализации рекомендовать. Всё техническое: хранение, трассировка, проверка зависимостей, генерация диаграмм, расчёт Value Score, форматирование документов — берёт на себя AIналитик.

Практический результат: BA, прошедший Главу 7 через платформу, передаёт в разработку не таблицу в Excel с 60 строками, а структурированный пакет: трассируемые верифицированные требования, обоснованный выбор варианта реализации, зафиксированные критерии успеха. Это профессиональная работа, которая экономит время всей команды и повышает вероятность что проект достигнет своих целей.